POLITECNICO MILANO 1863

DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

0.4

Best Bike Paths Requirements Analysis and Specification Document

MSc in Computer Science, Software Engineering 2
Academic year 2025-26

Professor: Matteo Giovanni Rossi

Authors:
Brini Luca, 10765193
Maiorana Martina, 10766914
Lucariello Francesco, 12345678

Revision History

Version	Date	Description
1.0	DD/MM/YYYY	Initial release

Contents

List of Figures

List of Tables

1 | Introduction

1.1. Purpose

The Best Bike Paths (BBP) system addresses the recurring need for Bike Riders to discover new routes and track their cycling activities. Registered users can record their rides either manually, by entering street names, or automatically, leveraging their device sensors.

All users, regardless of registration status, can search for paths between any given origin and destination, visualizing them on the map. Search results are ranked based on a score that integrates the path's status and its efficiency, measured as trip time. Additionally, the system offers statistics for each path to any user, including trip time and, when available, it also provides the current weather conditions along the route.

A registered user can access his previously recorded and validated trips and look at its performance, speed, and weather conditions during the recording.

The system aggregates identical paths submitted by multiple users, using metrics like freshness and status to combine the information. As part of this merging process, the final trip time is calculated by averaging the individual trip times of the paths being aggregated.

1.2. Scope

1.2.1. Description of the System

Best Bike Paths (BBP) is a standalone software meant to help cyclists facilitate the creation and management of a collective bike paths inventory. The software operates as a distributed system (like a downloadable smartphone application) that integrates both user-generated content and automated sensor data retrieved from users' mobile device/devices.

The system provides accurate and up-to-date details regarding paths thanks to a data aggregation engine capable of merging conflicting information from multiple sources based on freshness and confirmation from users.

BBP functions as an independent entity for the user but it relies heavily on this ecosystem of mobile sensors and external APIs (weather forecast, map display and so on).

Main Features

Trip Recording

- Manual entry of bike paths with street-level details and path status
- Automatic tracking of rides using mobile device sensors, including distance, average speed and duration
- Ensuring continuous data collection and storage of the current trip even in areas with no connectivity, to be synchronized once the connection is restored

Path Discovery and Information

- Search for bike paths between specified locations
- Visualization of paths on map
- Display of path characteristics (total distance, trip time)
- Provision of current weather information for a selected route

Up to date Information

- Aggregation of path data from multiple users

Personal Analytics

- Storage and retrieval of a registered user cycling history

1.2.2. Goals

- [**G1**]: A registered user aims to record its cycling trips
- [**G2**]: A registered user wants to access their previously recorded bike trips
- [**G3**]: A registered user aims to receive statistics about its past trips, including average speed, trip time and, when available, meteorological conditions (at recording time)
- [**G4**]: A registered user wants to share information about a bike path it has recorded, including their status and the presence of obstacles, making this information available to the community (all users)
- [**G5**]: A User wants to search for bike paths given an origin and a destination point and visualize results on both a map
- [**G6**]: A User wants to retrieve up to date information about bike paths, their status and current weather conditions

1.2.3. World and Machine Phenomena

World Phenomena

1. A Registered User wants to go for a bike ride and record the trip.
2. A User wants to get all paths available given origin and destination point.
3. A registered user wants to get all his private paths.
4. A User wants to know information about a specific bike path, such as total track distance, average speed and status.
5. A registered user wants to record a bike path in manual mode.
6. A registered user wants to record a bike path in automatic mode.
7. A registered user wants to publish one of his private paths.
8. A user wants to add an obstacle on the path (within a certain time interval from the path).
9. A mobile device with sensors (GPS, accelerometer, gyroscope) collects GPS and motion data while the user is biking.
10. External weather data services provide information about environmental conditions.

Shared Phenomena - World Controlled

1. A user signs up to the system.
2. A registered user starts recording their trip (automated mode).
3. A registered user stops recording their trip (automated mode).
4. A registered user inserts their bike trip data, specifying name and status of each street (manual mode).
5. A registered user accesses their private bike paths.
6. A registered user publishes their trip.
7. A user searches for all paths available from an origin and a destination point.
8. A user consults the list of paths on the map.
9. A user selects one of the proposed paths.
10. A user confirms or denies the presence of a pothole (automatically detected by BBP).

Shared Phenomena - Machine Controlled

1. The system shows bike trip data acquired from mobile devices of the registered user (automated mode).

2. The system asks the registered user to confirm or correct the presence of a pothole.
3. The system shows on the map the bike path(s) between given origin and destination points.
4. The system rank the paths based on their score (involving path status and effectiveness)
5. The system displays consolidated path information, merging data provided by different users

1.3. Definitions, Acronyms, Abbreviations

1.3.1. Definitions

- **Registered user**: users that are already registered in the platform.
- **Guest user**: users that are not registered in the platform
- **User**: term used to indicate both registered and guest users interacting with the platform, they can be registered or not already registered.
- **Bike path**: term used to indicate a path where a proper bike track exists or where cars are rare and speed limits are compatible with the average speed of a bike.
- **Trip**: private ordered set of GPS coordinates or street names with weather data recorded by a Register user
- **Path**: public route between two points, resulting from the merge process against multiple Trips with same GPS coords and total distance.

1.3.2. Acronyms

- **RASD**: Requirements Analysis and Specification Document
- **API**: Application Programming Interface
- **BBP**: Best Bike Paths
- **GPS**: Global Positioning System
- **GDPR**: General Data Protection Regulation

1.3.3. Abbreviations

- **G**[**n**]: Goal number n
- **R**[**n**]: Requirement number n
- **D**[**n**]: Domain assumption/Dependency/Constraint number n
- **UC**[**n**]: Use Case number n

1.4. Reference Documents

- Project assignment document
- IEEE Standard for Software Requirements Specifications

1.5. Document Structure

This document is organized as follows:

- **Chapter 1 - Introduction**: Provides an overview of the document, including purpose, scope, goals, and definitions
- **Chapter 2 - Overall Description**: Describes the general factors affecting the product including product perspective, functions, user characteristics, and assumptions
- **Chapter 3 - Specific Requirements**: Contains all software requirements in detail including functional requirements, use cases, performance requirements, and quality attributes
- **Chapter 4 - Formal Analysis**: Presents the Alloy formal specification
- **Chapter 5 - Effort Spent**: Documents the work distribution among team members

2 | Overall Description

2.1. Product Perspective

Best Bike Paths (BBP) is a standalone software meant to help cyclists facilitate the creation and management of a collective bike paths inventory. The software operates as a distributed system (like a downloadable smartphone application) that integrates both user-generated content and automated sensor data retrieved from users' mobile device/devices.

Specifically BBP interacts with mobile hardware interfaces, leveraging GPS to reconstruct trip trajectories and inertial sensors (gyroscopes and accelerometers) to infer road surfaces conditions and detect anomalies such as potholes. Furthermore the system interfaces with external meteorological services to enrich trip data with real-time variables like temperature, wind-speed and raining conditions.

The system provides accurate and up-to-date details regarding paths thanks to a data aggregation engine capable of merging conflicting information from multiple sources based on freshness and confirmation from users.

BBP functions as an independent entity for the user but it relies heavily on this ecosystem of mobile sensors and external APIs (weather forecast, map display and so on).

2.2. Product Functions

- **User Management**: Allows unregistered users to sign up and registered users to log in, managing their profiles and access rights to restricted features like data recording and publishing.
- **Automated Activity Tracking**: Automatically detects when a registered user is cycling based on mobile sensors' data. It records the trajectory via GPS and continuously monitors device sensors (accelerometer, gyroscope) to identify road anomalies and significant movements.
- **Manual Activity Tracking**: Enables registered users to manually insert bike path information by specifying street names and assigning them a status (e.g., "optimal", "requires maintenance") without real-time tracking.

- **Anomaly detection and user validation**: Processes sensor data to infer the presence of obstacles (e.g., potholes). It presents these potential anomalies to the user for post-trip validation to avoid false positives.
- **Data Enrichment**: Integrates with external weather services to enrich trip data with weather conditions (e.g., temperature, wind speed). This involves providing historical weather conditions to past trips and real-time weather conditions when a user visualizes a specific path.
- **Data Merging**: Implements a logic engine that fuses information about the same path from different users. It resolves conflicts and determines the current path status based on freshness and number of confirming data obtained.
- **Path Scoring and Searching**: Calculates a "Path Score" for available routes, combining the status and trip time. It allows users to search for paths between an origin and a destination, visualizing them on a map ranked by this score.
- **Statistical Reporting**: Generates performance metrics for registered users, offering the possibility to review past trips with data such as average speed and total trip time.

2.3. User Characteristics

In the BBP platform, a User can be a Guest User or a Registered User.

2.3.1. Guest User

A Guest User can explore bike paths without creating an account.

He can search for paths between any chosen origin and destination and view the corresponding routes directly on the map, selecting the safest or most efficient one. In fact, if multiple paths exist, BBP shows them to the user ranked by a score based on the path status and the effectiveness of the path in reaching the destination from the given origin. For each proposed path the Guest User can inspect its public characteristics, such as total distance, estimated trip time, and the status validated by the community.

Guest Users benefit from always-updated information, since BBP merges multiple contributions provided by different users to determine the most reliable and fresh status of each path.

Guest Users have read-only access to all public information collected by the system and cannot record trips, validate anomalies, or store personal path histories. Their interaction with BBP mainly focuses on quickly retrieving reliable, up-to-date information about the best available routes for their ride.

2.3.2. Registered User

A Registered User can access all the features available to Guest Users, but he can also access more advanced functionalities and actively contribute to the BBP system. In particular, he can record his trips and store them on the platform, keeping track of his cycling activity.

He can record his trips in two different ways:

- **Manual mode**: by inserting manually the names of the streets belonging to a bike path and assigning a status to each segment (such as "optimal", "medium", "sufficient", "requires maintenance").
- **Automatic mode**: letting BBP acquire data from their mobile devices, detecting when he starts cycling, collecting GPS data to reconstruct the followed path, and using accelerometer and gyroscope data to identify potential anomalies such as potholes. After the trip, the Registered User must confirm or reject the automatically detected anomalies within 24 h, before they become publishable.

A Registered User can review his personal cycling history, accessing stored trips enriched with statistics such as average speed, trip time and, when available, weather conditions at the time of recording.

When desired, he can publish one of his validated bike paths, including information such as total distance, estimated trip time, and the status), making it available to all other users.

2.4. Assumptions, Dependencies and Constraints

2.4.1. Domain Assumptions

- **[D.1.1] Sensor Reliability**: It is assumed that GPS, accelerometer and gyroscope sensors provided by the mobile hardware are sufficiently precise to distinguish biking vibrations from road anomalies effectively.
- **[D.1.2] Biking Speed Distinctiveness**: It is assumed that the speed and the acceleration of a user cycling are mathematically distinguishable from those of walking, running or driving a car, enabling the Automated Mode to function correctly without explicit user triggers.
- **[D.1.3] User Trustworthiness**: Given the community-based nature of the data, it is assumed that Registered Users will generally provide truthful validations regarding obstacles and path status.

2.4.2. Dependencies

- **[D.2.1] External Meteorological Service**: The system relies on third-party weather APIs to retrieve environment conditions. The availability and accuracy of "weather-enriched" statistics depend solely on these external provider/s.
- **[D.2.2] Mapping and Geocoding Service**: The visualization of paths and the translation of GPS coordinates to street names (Reverse Geocoding) depend on external map providers (e.g., Google Maps, OpenStreetMap).

- [**D.2.3**] **Mobile Hardware Ecosystem**: The application is strictly dependent on the user's device having functional hardware sensors and an OS that grants the necessary permissions for background tracking.

2.4.3. Constraints

- [**C.1**] **Network Connectivity**: A continuous network connection is required to access maps, fetch weather data and synchronize shared path information. Offline functionality is constrained to local buffering of the current trip recording.
- [**C.2**] **Power Consumption**: The continuous polling of GPS and motion sensors is resource-intensive. The software implementation must be optimized to prevent excessive battery drainage, which would hinder the user's ability to record long trips.
- [**C.3**] **Regulatory Compliance**: The storage of geolocation data and user habits is subject to strict privacy regulations. The system must enforce explicit consent mechanisms for data tracking and sharing.

2.5. Scenarios

1. **Account registration**: Unregistered user Mario accesses the sign-up interface to create a profile. Upon successful validation of the provided credentials, the system grants the new user the "Registered" status, allowing him to access new functions reserved to Registered Users.
2. **User login**: The registered user Clara inputs his credentials in BBP which validates them and loads the user's personal session.
3. **Automated trip tracking**: The registered user Clara starts cycling; the system detects the biking speed and automatically starts the recording of the trip via GPS, removing the need for manual interactions to start or stop the activity.
4. **Anomaly validation**: After an automated trip, the user Clara reviews the data collected via sensors by BBP and confirms or rejects potential potholes or obstacles detected to prevent false positives from polluting the database.
5. **Deferred Trip Data Publishing**: The registered user Clara accesses her personal history to select a trip recorded and validated in the past (e.g, days or months earlier) to share the collected data with the community.
6. **Performance statistics review**: The registered user Clara accesses her personal dashboard to view the historical statistics for each past trip, including average speed and trip duration.
7. **Weather-enriched statistics**: The registered user Clara accesses her personal dashboard to review her performance on one of her past trips where BBP has automatically fetched and added meteorological information (e.g., wind speed and temperature).

8. **Manual Path Insertion**: The Registered User Paolo manually inserts information about a specific bike path by writing down road names and assigning them a status (e.g., "optimal", "requires maintenance") without physically travelling the route at that moment.

9. **Path search and routing and score based visualization**: The user Sofia (can be registered or not) enters an origin and a destination. BBP calculates and visualizes the available paths by connecting the two points on a map. When multiple routes exist for a search query, the user Sofia views them ranked by a "Path score" with real-time weather conditions.

10. **Path score and freshness calculation**: The registered user Paolo reports that a bike path in Via Venezia is "optimal." Two days later, another registered user Clara rides along the same path, and the system automatically detects bumps suggesting possible degradation. Shortly after, the registered user Sofia manually marks the same path as "requires maintenance." BBP collects all user contributions, compares their freshness, and resolves conflicting information, updating the path's status.

2.6. Class Diagram

The UML Class diagram represents a high level model of the mobile application described in this document. There are two kinds of users: RegisteredUser and its superclass User. Trip class represents a bike ride of a RegisteredUser, it can be an AutomaticTrip or a ManualTrip. Whenever a Trip is published by its creator, the system merges it with other Trip(s) from different RegisteredUser(s), contributing to the public Path, which can be searched, found and seen by User(s). The ManualTrip class has a list of streets, which are keyed in by the RegisteredUser. They are suddenly translated into GPSCoordinates by using an external MapService, also used for Street lookup and Path visualization. On the other hand, AutomaticTrip leverages the MobileSensors for directly obtaining GPSCoordinates and SensorReadings which are analyzed to find the Obstacle(s). Any Obstacle must be confirmed by the RegisteredUser recording the Trip before it can be published. Statistics depending on speed, such as average speed and trip time, are only recorded for AutomaticTrip. Weather conditions are provided by an external WeatherService and are used within Statistics since they can influence performance. Also User consults the WeatherService when it views a public Path.

Figure 2.1: Class Diagram

2.7. State Diagrams

2.7.1. Path State Diagram

This state diagram shows how the status of a Path evolves as BBP receives publishable information from different Registered Users. When a path has no community data, it is in the "No Data" state. As soon as the first publishable Trip is merged, the Path enters one of the status states (e.g., "Optimal" or "Requires Maintenance"). Every time a new Trip about the same trajectory is published, the merging engine re-evaluates the status using freshness and the number of confirming contributions. If recent reports agree on a different status, the Path moves to the corresponding state (e.g., from "Optimal" to "Requires Maintenance"). When all reliable information indicates that the path can no longer be used, it moves to the "Permanently Closed" final state.

Figure 2.2: Path State Diagram

2.7.2. Manual Trip State Diagram

This state diagram describes the lifecycle of a ManualTrip. When a Registered User decides to insert a path manually, the trip enters the "Creating" state while the user specifies the list of streets and assigns a status to each segment. Once the trip is saved, it becomes part of the user's private history in the "Saved (Private)" state, and it can still be edited, going back to "Creating" if needed. When the user decides to share the collected information with the community, the trip moves to the "Published" state and contributes to public paths. At any time, the user may delete the trip, which moves it to the "Deleted" final state.

Figure 2.3: Manual Trip State Diagram

2.7.3. Automatic Trip State Diagram

This state diagram captures the evolution of an AutomaticTrip. While the user is not cycling, the trip is in the "Idle / Not Recording" state. When BBP detects biking activity through speed and motion sensors, or when the user explicitly starts automatic recording, the trip moves to the "Recording" state, where GPS coordinates and sensor readings are collected. When recording stops, if the system has detected potential obstacles, the trip enters the "Awaiting Validation" state: the Registered User has 24 hours to confirm or reject each obstacle. Once all obstacles are validated (or the timeout expires), the trip becomes "Ready to Publish". When the user chooses to share it with the community, the trip reaches the "Published" state and contributes to public path information; otherwise, the trip can be discarded, reaching the final "Discarded" state.

Figure 2.4: Automatic Trip State Diagram

3 | Specific Requirements

3.1. External Interface Requirements

3.1.1. User Interfaces

- **Sign Up/Login Page**: The system must include a Sign Up/Login page that allows Users to register and authenticate their credentials for accessing the platform. This functionality ensures secure access for registered users.
- **Home Page and Search Bar**: The system must include a home page which has in the bottom buttons to access personal dashboard and profile management and in the top should have a search bar to insert origin and destination points of a trip.
- **Map interface**: When Users search a path, a map interface representing the possible paths, ordered by score, should be presented to the user. Users can tap on various paths and look at some details.
- **Dashboard**: The system must include a dashboard section where registered users can look at all their past registered trips and related statistics (e.g., total distance, trip time, average speed).

3.1.2. Hardware Interfaces

The Best Bike Paths (BBP) application interacts directly with the following hardware components hosted on the user's device:

- **GPS Receiver**: The system shall interface with the GPS receiver to obtain real-time geolocation coordinates for trip tracking and navigation.
- **Inertial Sensors**:
  - **Accelerometer**: Used to measure acceleration to detect sudden jolts indicative of potholes.
  - **Gyroscope**: Used to measure orientation and angular velocity to distinguish between road anomalies and phone handling movements.
- **Touchscreen Display**: Used for all user inputs and visual output.

3.1.3. Software Interfaces

The system interacts with the following external software services and APIs:

- **Map Rendering Service**: The system shall interface with an external mapping provider (e.g., Google Maps Platform, OpenStreetMaps) to retrieve map tiles and perform geocoding services.
- **Meteorological Data Service**: The system shall communicate with a third-party weather API to fetch real-time environmental data (or past for registered trips) based on the user's location.

3.1.4. Communication Interfaces

The application can be accessed through a stable internet connection to perform all actions and access all functionalities. Connection is not required while biking since local buffering for trip recording is present.

3.2. Functional Requirements

3.2.1. Sign up and log in

- [**R1**]: The system must allow an unregistered User to sign up
- [**R2**]: The system must allow registered Users to login

3.2.2. Manual Trip Tracking

- [**R3**]: The system must allow a Registered User to manually insert a bike path by listing the name and the status of each street comprising the path.
- [**R4**]: The system must store manually inserted paths in the User's private trip history.

3.2.3. Automatic Trip Tracking

- [**R5**]: The system must allow a Registered User to automatically record a trip, using device sensors to detect when he starts and stops biking.
- [**R6**]: The system must automatically identify the start of a Registered User's biking activity and initiate the recording process.
- [**R7**]: The system must automatically start GPS-based tracking when biking is detected.
- [**R8**]: The system must collect GPS coordinates to reconstruct the path followed by the User.
- [**R9**]: The system must collect accelerometer and gyroscope data to detect potential anomalies (e.g., potholes).

- [**R10**]: The system must store complete trip statistics (distance, average speed, duration and, when available, weather conditions at the time of recording).
- [**R11**]: The system must allow a registered user to view the complete statistics (distance, average speed, duration and, when available, weather conditions) related to his automated-recorded trip.
- [**R12**]: The system must buffer trip data locally when network connectivity is unavailable and must synchronize buffered trip data once connectivity is restored.
- [**R13**]: The system must detect when biking ends and automatically stop tracking.
- [**R14**]: In case of anomalies detection, the system must allow a Registered User to confirm or reject each detected anomaly within 72 hours from the trip recording.
- [**R15**]: The system must allow a Registered User to publish a validated trip.

3.2.4. Trip Storage and Personal Analytics

- [**R16**]: The system must allow Registered Users to store all their recorded trips (manually or automatically).
- [**R17**]: The system must allow a Registered User to view his private trip history, including relatively collected information.
- [**R18**]: The system must allow a Registered User to review the statistics related to a bike path, such as average speed, and other performance metrics.

3.2.5. Path Search, Routing and Visualization

- [**R19**]: The system must allow any user to access published paths
- [**R20**]: The system must allow any User to search the bike path(s) between a given origin and a given destination.
- [**R21**]: The system must visualize the paths on a map interface.
- [**R22**]: The system must compute a Path Score based on path status and estimated trip time.
- [**R23**]: The system must rank available paths based on their Path Score.
- [**R24**]: The system must allow a User to inspect public details of a selected path, such as status, total distance, trip time, and validated anomalies.
- [**R25**]: The system shall query an external weather service to retrieve current weather conditions during the path search and display relevant warnings to the user.

3.2.6. Merging & Freshness-Based Status Aggregation

- [**R26**]: The system shall append historical weather data (temperature, wind speed and direction) to the trip logs of registered users for performance analysis.
- [**R27**]: The system must merge publishable information from multiple Users referring to the same path, considering their freshness and the number of confirming contributions.

3.3. Use Cases

3.3.1. Use Case Diagram

Figure 3.1: Use Case Diagram

3.3.2. Use Case Descriptions

[UC1] - Sign Up

Participating Actors	Guest User
Entry Conditions	The Guest User is not authenticated and wants to become a Registered User
Flow of events	1. The Guest User clicks on "sign up" 2. BBP displays the registration form 3. The User inputs email, username, password 4. The User clicks "create account" 5. BBP validates the input syntax 6. BBP creates a new profile in the database 7. BBP confirms creation and redirects to the Login Page
Exit Conditions	The User has successfully created a new account.
Exceptions	E-mail already in use: BBP notifies the User and suggests logging in E-mail not valid: BBP notifies the User and suggest correcting it

Table 3.1: Use Case 1: Sign Up

Figure 3.2: Sequence Diagram for UC1: Sign Up

[UC2] - Log In

Participating Actors	Registered User
Entry Conditions	The Registered User is not currently authenticated
Flow of events	1. The User clicks on "login" 2. BBP displays the login form 3. The User inputs their credentials (email/username, password) 4. The User clicks the "login" button 5. BBP hashes the password and validates it against the database 6. BBP retrieves the user's profile and starts a session 7. BBP redirects the user to the Home Page
Exit Conditions	The User is authenticated and can access restricted features.
Exceptions	Invalid Credentials: BBP displays an error message and invites the user to retry.

Table 3.2: Use Case 2: Log In

Figure 3.3: Sequence Diagram for UC2: Log In

[UC3] - Record Trip Automatically

Participating Actors	Registered User, Mobile Sensors, Weather Service
Entry Conditions	The Registered User is ready to automatically record his biking activity
Flow of events	1. User clicks on "Start automatic recording" 2. BBP enters detection mode and starts monitoring GPS speed 3. BBP detects biking activity (speed $>$ threshold) 4. BBP starts the recording session 5. BBP continuously logs GPS coordinates and timestamps (trajectory) 6. BBP monitors accelerometer/gyroscope for anomalies (e.g., potholes) Alt: speed $<$ threshold BBP pauses data acquisition and returns to step 2 7. User presses "End Automatic Record" button 8. BBP stores the detailed raw data (GPS, trajectory, timestamps) 9. BBP retrieves the historical meteorological data from the Weather Service and link it with the trip 10. BBP analyzes sensors data to identify potholes and stores them as PENDING 11. BBP calculates the specific statistics (total distance, average speed, and duration) and stores them 12. BBP saves the trip locally 13. BBP synchronizes the trip with the server
Exit Conditions	A new trip is recorded with its potholes, statistics and weather conditions
Exceptions	Connectivity Loss: BBP buffers data locally until connection is restored.

Table 3.3: Use Case 3: Record Trip Automatically

Figure 3.4: Sequence Diagram for UC3: Record Trip Automatically

[UC4] - Record Trip Manually

Participating Actors	Registered User, Map Service
Entry Conditions	The Registered User is ready to manually insert trip information
Flow of events	1. The user selects "Manual Record" 2. BBP presents a form to insert a street name Loop The user specifies a street name (a) The BBP queries the Map Service with provided street name (b) The BBP displays found streets to the user (c) Registered User selects one of the found streets End Loop BBP add the chosen street to the trip 3. The user submits the form 4. BBP confirms the submission 5. BBP saves the data
Exit Conditions	A new manual trip is recorded
Exceptions	Invalid Location: The system cannot identify the street provided. abort Map Service is unavailable: BBP shows an error regarding the map service and cancel the operation suggesting trying again later

Table 3.4: Use Case 4: Record Trip Manually

Figure 3.5: Sequence Diagram for UC4: Record Trip Manually

[UC5] - Validate Obstacle

Participating Actors	Registered User, Map Service
Entry Conditions	The registered user is on its personal history page and is ready to validate an automatically recorded Trip
Flow of events	1. The Registered User chooses one of its automatically recorded private trips 2. BBP highlights potential selected trip's anomalies on the map 3. The user reviews a specific anomaly Alt The user confirms the detection Alt The BBP updates pothole status to CONFIRMED Alt2 The user rejects the detection Alt2 The BBP updates pothole status to REJECTED 4. BBP updates the trip accordingly 5. The user saves the validated trip
Exit Conditions	Trip obstacles are validated and trip is ready to for optional publication
Exceptions	

Table 3.5: Use Case 5: Validate Obstacle

Figure 3.6: Sequence Diagram for UC5: Validate Obstacle

[UC6] - Publish Trip

Participating Actors	Registered User
Entry Conditions	The registered user has at least one private trip ready to publish
Flow of events	1. The user selects a private trip from his history 2. The user clicks "publish" 3. BBP changes the trip status to "public" 4. BBP anonymizes trip data 5. BBP triggers the Data Merging algorithm to integrate anonymized data with already existing data (considering freshness/confirmations) 6. BBP updates the global path score 7. BBP confirms publication
Exit Conditions	Trip data has been integrated in public path and it's available to all users
Exceptions	

Table 3.6: Use Case 6: Publish Trip

Figure 3.7: Sequence Diagram for UC6: Publish Trip

[UC7] - View Private Trip

Participating Actors	Registered User, Map Service
Entry Conditions	The registered user has recorded at least one trip and wants to view it in details
Flow of events	1. The user clicks on "My Trips" 2. BBP displays the list of the Registered User trips over time 3. The user selects a specific trip from his history list 4. BBP fetches map data about chosen trip's coordinates 5. BBP displays a detailed view showing trip trajectory on the map, computed statistics, found/confirmed potholes and weather conditions faced during the ride
Exit Conditions	The user successfully visualizes the detailed trip performance and context of a single specific trip
Exceptions	Map Service is unavailable: BBP shows an error regarding the map but still shows all other things

Table 3.7: Use Case 7: View Private Trip

Figure 3.8: Sequence Diagram for UC7: View Private Trip

[UC8] - Delete Private Trip

Participating Actors	Registered User
Entry Conditions	The registered user has recorded at least one trip and wants to delete it
Flow of events	1. The registered user clicks on "My Trips" 2. BBP displays the list of the user trips over time 3. The user keeps pressed on a specific trip from his history list 4. BBP shows an additional in-context menu 5. The user selects the "Delete" option 6. BBP ask for confirmation Alt The user confirm the operation Alt BBP deletes the selected Trip from User history Alt The user cancels the operation Alt BBP aborts the operation and displays the user trips history
Exit Conditions	Trip is removed from the registered user's private history. If previously published, anonymized data remains in aggregated PATH information.
Exceptions	Invalid Location: The system cannot identify the street provided. abort Map Service is unavailable: BBP shows an error regarding the map service and cancel the operation suggesting trying again later

Table 3.8: Use Case 8: Delete Private Trip

Figure 3.9: Sequence Diagram for UC8: Delete Private Trip

[UC9] - Search Path

Participating Actors	Any User (Guest/Registered), Map Service
Entry Conditions	The User is ready to search paths between two points
Flow of events	1. The user clicks on the search bar 2. BBP displays search form Loop for Origin and Destination endpoints (a) The user enters endpoint street name (b) BBP queries the Map Service with provided street name (c) BBP displays found streets to the user (d) The User selects one of the found streets (e) BBP add selected street as endpoint value End Loop 3. BBP searches paths with provided endpoints 4. BBP ranks the found paths by previously computed/updated score 5. BBP shows the ranked list of paths to the user
Exit Conditions	User is shown of available paths ready to be viewed in details
Exceptions	Invalid Location: The system cannot identify the street provided. abort Map Service is unavailable: BBP shows an error regarding the map service and cancel the operation suggesting trying again later No Paths Found: BBP shows an error message saying "Path not found" and await for the user to return back or changing query

Table 3.9: Use Case 9: Search Path

Figure 3.10: Sequence Diagram for UC9: Search Path

[UC10] - Visualize Path

Participating Actors	Any User (Guest/Registered), Map Service, Weather Service
Entry Conditions	The User has searched paths between two points and has chosen one listed option
Flow of events	1. The user clicks on one path in the search results list 2. BBP fetches map data about chosen path's coordinates 3. BBP queries external Weather Service about current conditions along the chosen path 4. BBP displays a detailed view showing path trajectory on the map, aggregated statistics (average time, total distance and status), confirmed potholes and current weather conditions along the path
Exit Conditions	User is shown of path on the map, its aggregated statistics and current weather conditions
Exceptions	Weather Service is not available: BBP shows an error regarding weather conditions but still loads everything else Map Service is unavailable: BBP shows an error regarding the map but still shows all other things

Table 3.10: Use Case 10: Visualize Path

Figure 3.11: Sequence Diagram for UC10: Visualize Path

3.4. Performance Requirements

To ensure an optimal user experience, the system should complete actions within 2 seconds. However for this application, that does not involve critical services, strict requirements are not needed. For this reason completion time for complex tasks can be extended up to 5 seconds. Those complex tasks could be:

- searching and displaying available paths
- calculating and displaying statistical information

3.5. Design Constraints

3.5.1. Standards Compliance

Since the system collects and stores sensitive user data, the design must strictly adhere to the General Data Protection Regulation (GDPR). This includes implementing mechanisms for:

- Explicit user consent for tracking.

- The deletion of all user data upon request.
- Anonymization of data when aggregated for public statistics.

3.5.2. Hardware Limitations

- **Mandatory Sensor Availability**: The automated tracking module is constrained by the physical hardware of the user's device. The application cannot function in automated mode on devices lacking working sensors such as GPS, accelerometer, gyroscope.
- **Local Storage Capacity**: The local buffering feature for offline recording is constrained by the available internal storage of the mobile device. The application design must include a safe upper limit for the buffer size to avoid filling the user's device storage.
- **Battery Drain Constraints**: The software must use optimizations techniques or/and APIs to minimize battery usage.

3.5.3. Implementation Constraints

- **Third-party APIs Limits**: The design is constrained by the Rate Limits imposed by external service providers. The software must implement some strategies to avoid exceeding Rate Limits and incurring service denials or extra costs.

3.6. Software System Attributes

3.6.1. Reliability

To ensure better reliability performance, regular maintenance and bug fixes should be performed. Planned maintenance should also be notified to users to avoid any unexpected inconvenience.

3.6.2. Availability

To ensure a positive user experience, the system uptime should be 99% or more. This application does not involve critical tasks (e.g., financial transactions) so the minimum standard it's not required to be 99.5% or more.

The core function of trip recording in automated mode should be available 100% of the time, regardless of network status, provided the device has sufficient battery and storage.

3.6.3. Security

The system shall enforce strict access control policies. A user's specific history and sensor logs must be accessible only to that specific user, unless explicitly published.

User passwords shall never be stored in plain text. The system must use industry-standard hashing algorithms with unique salts for credential storage.

3.6.4. Maintainability

The architecture should follow a modular approach to facilitate debugging and programming.

The source code should also be documented following standard conventions (e.g., Javadoc for Java, PyDoc for Python) to facilitate future maintenance and onboarding of new developers.

3.6.5. Portability

BBP is a mobile application so it has to be compatible with both major OSs (Android and iOS).

4 | Formal Analysis Using Alloy

4.1. Introduction to the Model

This chapter presents the formal specification of the system using Alloy, a declarative modeling language based on first-order logic. The model captures the key entities, their relationships, and the state transitions described in the previous chapters.

4.2. Alloy Code

Listing 4.1: BBP Formal Model

```
module bbp_formal

// Enumerations
// track the status of a Public Path
enum PathStatus {
    Optimal, Medium, Sufficient, RequiresMaintenance,
       PermanentlyClosed
}

enum PathLifecycle{
    Created, MergeData
}

// track the lifecycle of a manually-recorded Trip
enum ManualTripState {
    InsertingStreet, SavedPrivate, ReadyToPublish, Published,
       Deleted
}

// track the lifecycle of an automatic-recorded Trip
enum AutomaticTripState {
    Idle, DetectBiking, Recording, Analysis, SavedPrivate,
       AwaitingValidation,
    ReadyToPublish, NotPublishable, Published, Deleted
}

```

```
// Entities
sig User {} // Guest or Registered (we model registration via a
    subset)

sig RegisteredUser in User { // Registered users are a subset of
    all users
    recorded: set Trip
}

// A public path whose status evolves over time
sig Path {
    var status: one PathStatus
}

// abstract signature Trip with two subtypes (ManualTrip and
    AutomaticTrip)
abstract sig Trip {
    owner : one RegisteredUser, // only Registered Users can
        record a Trip
    about : one Path,
    reportedStatus : one PathStatus // what the user says about
        the path
}

sig ManualTrip extends Trip {
    var mState : one ManualTripState
}

sig AutomaticTrip extends Trip {
    var aState : one AutomaticTripState
}

// Ensures that all the Trip were recorded by exactly one
    Registered User
// and 'recorded' matches that ownership
fact recordedMatchesOwner {
    all t: Trip | t in t.owner.recorded
    all u: RegisteredUser | u.recorded = { t: Trip | t.owner = u
        }
}
```

4.3. Model Verification

The model can be executed using Alloy Analyzer to verify various properties of the system, such as:

- Ensuring that only registered users can create trips

- Verifying that the state transitions follow the defined state diagrams
- Checking that path status updates correctly based on merged trip data

5 | Effort Spent

This chapter documents the time each team member dedicated to the development of this RASD document.

5.1. Brini Luca

Activity	**Hours**
Chapter 1	6
Chapter 2	3
Chapter 3	15
Chapter 4	–
Chapter 5	–
Total	**24**

Table 5.1: Effort Spent - Brini Luca

5.2. Maiorana Martina

Activity	**Hours**
Chapter 1	4
Chapter 2	2
Chapter 3	–
Chapter 4	–
Chapter 5	–
Total	**6**

Table 5.2: Effort Spent - Maiorana Martina

5.3. Lucariello Francesco

Activity	**Hours**
Chapter 1	–
Chapter 2	3
Chapter 3	–
Chapter 4	–
Chapter 5	–
Total	**3**

Table 5.3: Effort Spent - Lucariello Francesco